

Quest3 Profile 구현 설명서

브랜치: `Josh\_No\_AI`

대상 모듈: `module/quest3Tier`

작성 목적: 코드 리뷰 및 발표에서 Profile 처리 흐름, 변경 이유, 코드 위치를 정확히 설명하기 위한 학습 문서

1. 요구사항

Jake와 확인한 요구사항은 다음과 같다.

1. 프론트엔드는 Profile 초기화 API를 호출하지 않는다.
2. 인증이 필요한 Quest3 요청은 공통 `requestHandler` wrapper를 통과한다.
3. wrapper는 Microsoft JWT의 `oid`를 읽는다.
4. `oid`를 `Profile.external\_id`로 사용해 Quest3 Profile을 찾는다.
5. 같은 `external\_id`에 Profile이 여러 개면 현재는 첫 번째 Profile을 사용한다.
6. Profile이 없으면 새 Profile을 생성한다.
7. downstream handler에는 선택된 Profile의 `internal\_id`를 `request.userData.profileId`로 전달한다.

2. 두 ID의 차이

external_id

- 출처: Microsoft Entra ID JWT의 `oid` claim
- 의미: Microsoft 사용자를 식별하는 외부 시스템 ID
- 용도: Quest3 Profile을 검색하는 키
- 하나의 `external\_id`가 여러 Quest3 Profile과 연결될 수 있다.

internal_id

- 출처: Quest3 데이터베이스가 UUIDV4로 생성
- 의미: Quest3 내부 Profile의 primary key
- 용도: Question, QuestionAnswer, QuestionShare 등의 `profileId` 외래 키

예시:

```
```text
Microsoft oid (external_id): E
├─ Quest3 Profile A (internal_id: A)
└─ Quest3 Profile B (internal_id: B)
```

현재 선택 결과: 먼저 생성된 Profile A  
downstream에 전달되는 값: A

`external\_id`와 `internal\_id`는 우연히 같은 UUID 문자열일 수는 있지만 의미가 다르므로 코드에서 같은 안 된다.

### ## 3. 전체 요청 흐름

```
```text
Frontend jwtFetch
  ↓ Authorization: Bearer <token>
Azure Function route
  ↓
requestHandler wrapper
├─ JWT 검증
├─ decoded.oid → externalId
├─ ensureProfile(externalId)
│   └─ Profile.external_id로 검색
```

```

      |      └─ 있으면 첫 번째 Profile 반환
      └─ 없으면 새 Profile 생성
    └─ profile.internal_id → request.userData.profileId
↓
CreateQuestion / AddAnswer / ShareQuestion 등의 handler
↓
service 및 database

```

프론트엔드는 Profile을 미리 생성할 필요가 없다. 첫 번째 실제 인증 요청이 wrapper를 통과할 때 Profile이 생성된다.

4. 파일별 상세 설명

4.1 Profile 모델

파일: `module/quest3Tier/func/repository/model/Profile.mjs`

- 10~15행: `internal\_id`는 primary key이며 UUIDV4 기본값으로 자동 생성된다.
- 16~19행: `external\_id`는 Microsoft `oid`를 저장하며 null은 허용하지 않는다.
- `external\_id`의 unique 설정을 제거했다. 한 Microsoft 사용자에게 여러 내부 Profile을 허용하기 위함이다.
- `updatedAt: false`이므로 Sequelize는 `createdAt`은 관리하지만 `updatedAt` 컬럼은 사용하지 않는다.

4.2 Profile 조회 및 생성 서비스

파일: `module/quest3Tier/func/service/profile.mjs`

- 6행: DI container에서 등록된 모델을 사용한다.
- 9행: `ensureProfile(externalId)`가 wrapper에서 호출되는 진입점이다.
- 10~14행: ID가 없거나 UUID가 아니면 status 401인 오류를 던진다.
- 16행: DI container에서 `Profile` Sequelize 모델을 얻는다.
- 17~23행: `external\_id`로 기존 Profile 하나를 조회한다.
- 19~22행: `createdAt ASC`, 그 다음 `internal\_id ASC`로 정렬한다. “첫 번째”가 요청마다 달라지므로 정적인 순서를 정의한 것이다.
- 25~27행: 기존 Profile이 있으면 재사용하고 `created: false`를 반환한다.
- 29행: 기존 Profile이 없으면 `external\_id`만 제공해 생성한다. `internal\_id`는 모델의 UUIDV4 기본값으로 생성된다.
- 30행: 생성된 Profile과 `created: true`를 반환한다.

왜 `internal\_id`로 조회하지 않는가? wrapper가 가진 값은 Microsoft `oid`, 즉 외부 ID이기 때문이다.

왜 생성할 때 `internal\_id`를 넣지 않는가? 내부 ID 생성은 Quest3 데이터 모델의 책임이고 Microsoft ID를 생성할 필요가 없다.

4.3 인증 wrapper

파일: `module/quest3Tier/func/handler/handlerWrapper.mjs`

- 43행: 기본적으로 인증이 필요한 요청만 Profile 로직을 실행한다.
- 44~51행: 인증 provider와 Authorization header에서 JWT를 얻는다.
- 52행: provider가 token을 검증하고 decode한다.
- 53행: `decoded.oid`를 `externalId`라는 이름으로 저장한다. 이 이름은 Microsoft 외부 ID라는 의미이다.
- 54행: `ensureProfile(externalId)`를 호출한다.
- 55행: 선택된 `profile.internal\_id`를 downstream에서 사용할 `profileId`로 정한다.
- 56행: `profileId`, `externalId`, Profile record, 생성 여부를 `user` 객체에 저장한다.
- 59행: `request.userData`에 user 객체를 넣는다.
- 이후 실제 handler가 실행된다.

가장 중요한 줄은 다음 관계다.

```

```text
decoded.oid → externalId

```

`profile.internal_id → request.userData.profileId`

### ### 4.4 Downstream handlers

파일: ``module/quest3Tier/func/handler/handler.mjs``

다음 작업들은 request body, header 또는 URL에서 현재 사용자의 ID를 믿지 않고 wrapper가 설정한 ``request.userData.profileId``를 사용한다.

- ``CreateQuestion``: 57~61행
- ``AddAnswer``: 함수 내부의 ``request.userData.profileId``
- ``ShareQuestionById``: sender ID로 ``request.userData.profileId`` 사용
- ``PatchQuestionById``: 변경 기록 작성자의 Profile ID로 사용
- ``SendFollowUpCmd``: 명령 발신자의 Profile ID로 사용
- ``ShareQuestionCmd``: 명령 발신자의 Profile ID로 사용
- ``GetQuestionListByUser``: 321~324행
- ``GetSharedQuestionListByUser``: 529~532행

특히 목록 조회 handler는 legacy URL에 ``{profileId}``가 남아 있더라도 그 값을 신뢰하지 않고 인증된 ``request.userData.profileId``를 사용한다. 이는 다른 사용자의 ID를 URL에 넣어 조회하는 것을 막는다.

### ### 4.5 DB migration

파일: ``module/quest3Tier/func/deploy/db/migration/20260726000000-link-profile-foreign-keys.mjs``

- Profile 테이블 생성 migration에서는 ``external_id``의 임의 UUID 기본값을 제거했다. 이 값은 반드시 ``rossoft`` ``oid``에서 와야 하기 때문이다.
- 기존 Question 관련 테이블의 Profile ID를 ``profile.internal_id``에 연결한다.
- 기존 데이터에서 사용 중인 Profile ID를 먼저 ``profile`` 테이블로 backfill한다.
- ``external_id`` unique constraint 생성 로직을 제거했다.
- ``down()``에서도 해당 unique constraint 삭제 로직을 제거했다.
- Profile이 참조 중이면 삭제를 막는 ``onDelete: RESTRICT``를 유지한다.

주의: 이 migration이 이미 공유 환경에 적용됐다면 파일 수정만으로 기존 DB의 unique constraint가 없어진 환경에는 constraint를 제거하는 후속 migration이 필요하다.

### ### 4.6 제거한 Profile 초기화 API

제거 위치:

- ``module/quest3Tier/func/route.mjs``: ``POST /profile`` route 제거
- ``module/quest3Tier/func/handler/handler.mjs``: ``EnsureProfile`` handler 제거
- ``module/quest3Tier/ui/api/question.ts``: ``ensureCurrentProfile()`` 제거
- ``module/quest3Tier/ui/AppRoutes.tsx``: 초기화 요청, ``profileReady``, ``profileError`` 제거

제거 이유:

- Profile 생성 책임이 프론트엔드와 wrapper에 중복돼 있었다.
- 프론트엔드가 초기화 API 호출을 빠뜨리면 다른 API가 실패할 수 있었다.
- 공통 wrapper에서 처리하면 모든 인증 요청에 동일한 규칙이 적용된다.
- Profile 준비를 기다리는 동안 UI가 빈 화면을 렌더링하던 로직도 불필요해졌다.

## ## 5. 보안상 의미

클라이언트가 보내는 Profile ID는 사용자가 수정할 수 있다. 따라서 현재 로그인 사용자를 결정할 때 다음 값들

- JSON body의 ``profileId``
- URL parameter의 ``profileId``
- ``x-profile-id`` header

- 브라우저 localStorage의 `profileId`

현재 사용자는 검증된 JWT의 `oid`에서 시작해 DB Profile로 변환해야 한다. 다른 사용자에게 질문을 공유할 때 `ds`는 대상 선택 데이터이므로 별도 개념이다.

## ## 6. 테스트 설명

파일: `module/quest3Tier/func/vitest/profile.test.mjs`

테스트는 다음을 검증한다.

1. Profile이 없으면 `external\_id`만 전달해 생성하고 DB가 만든 별도 `internal\_id`를 받는다.
2. Profile이 이미 있으면 첫 번째 Profile을 재사용하고 새 Profile을 만들지 않는다.
3. 외부 ID가 없으면 401 오류가 발생하며 DB를 호출하지 않는다.
4. 외부 ID가 UUID 형식이 아니면 401 오류가 발생하며 DB를 호출하지 않는다.
5. wrapper가 Microsoft `oid`와 다른 Quest3 `internal\_id`를 downstream handler에 전달한다

실행 명령:

```
``text
pnpm --filter quest3Tier test:profile
```

확인 결과: 테스트 파일 1개, 테스트 5개 모두 통과.

## ## 7. 현재 한계와 후속 작업

### ### Profile 선택 화면 없음

같은 Microsoft 사용자에게 Profile이 여러 개 있어도 현재는 가장 먼저 생성된 Profile을 자동 사용한다. 기면 wrapper는 사용자가 선택한 internal ID가 해당 external ID에 속하는지 검증해야 한다.

### ### 최초 동시 요청

같은 사용자의 최초 요청 두 개가 완전히 동시에 실행되면 둘 다 조회 결과가 없다고 판단해 Profile을 각각 생성 file을 허용하는 현재 모델에서는 DB 제약 위반은 아니지만 의도하지 않은 복수 Profile이 될 수 있다. 제품 요구사항, lock 또는 별도 매핑 정책을 검토해야 한다.

### ### Legacy URL

질문 목록 route에는 아직 `/profile/{profileId}/...` 형태가 남아 있다. handler는 URL 값을 무시하고 `request.userData.profileId`를 사용하므로 보안상 신뢰하지 않지만, API 의미를 명확히 하려면 향후 URL parameter를 제거할 수 있다.

### ### 기존 DB migration

수정 전 unique constraint가 이미 배포된 DB에는 별도 제거 migration이 필요하다. 배포 여부를 반드시 확인

## ## 8. 발표용 1분 설명

“Microsoft JWT의 oid는 외부 사용자 ID이고 Quest3 Profile의 internal\_id는 우리 도메인의 Profile ID입니다. 한 Microsoft 사용자에게 여러 Quest3 Profile이 연결될 수 있기 때문에 두 ID를 같게 저장하면 안 됩니다. handler wrapper가 JWT를 검증한 다음 oid를 external\_id로 사용해 Profile을 찾습니다. 여러 Profile이 있는 현재는 생성 순서상 첫 번째를 사용하고, 없으면 내부 UUID가 별도로 생성되는 새 Profile을 만듭니다. 이후 선택된 profile.internal\_id를 request.userData.profileId로 전달합니다. 따라서 프론트엔드가 별도 API 초기화 API를 호출하거나 body와 URL로 현재 사용자의 Profile ID를 지정할 필요가 없습니다.”

## ## 9. 예상 질문과 답변

### ### 왜 external\_id를 unique로 만들지 않았나?

한 Microsoft 사용자에게 여러 Quest3 Profile이 연결될 수 있다는 요구사항 때문이다.

### 왜 첫 번째 Profile을 정렬해서 찾나?

정렬이 없으면 DB가 반환하는 첫 행이 달라질 수 있다. 현재는 `createdAt ASC`, `internal\_id ASC`로 만들었다.

### 왜 프론트엔드 초기화 API를 없앴나?

인증된 모든 API가 공통 wrapper를 통과하므로 wrapper에서 조회/생성하는 것이 일관되고 초기화 호출 누락도

### 왜 body의 profileId를 사용하지 않나?

클라이언트 값은 사용자가 변경할 수 있어 현재 사용자의 신원을 증명하지 못하기 때문이다.

### receiverIds도 wrapper 값으로 바뀌어야 하나?

아니다. sender는 현재 로그인 사용자이므로 wrapper 값이고, receiverIds는 사용자가 공유 대상으로 선택한 file ID 목록이다.

### internal\_id는 누가 생성하나?

Sequelize Profile 모델의 `DataTypes.UUIDV4` 기본값이 생성한다. 서비스는 `external\_id`만 전달한다.